

Metody Obliczeniowe w Nauce i Technice

Laboratorium 6 – Kwadratury, część I

Mateusz Stoch

21 kwietnia 2026

1 Zadanie 1 – Złożone kwadratury otwarte

1.1 Opis zadania

Celem zadania jest numeryczne obliczenie całki:

$$\int_0^1 \frac{4}{1+x^2} dx = \pi$$

za pomocą trzech złożonych kwadratur Newtona-Cotesa: prostokątów (ang. mid-point rule), trapezów oraz Simpsona. Na przedziale całkowania należy rozmieścić $2^m + 1$ równoodległych węzłów, gdzie m przyjmuje wartości od 1 do 25. Oznacza to podwajanie liczby podprzedziałów w każdym kroku, w konsekwencji zagęszczając siatkę.

Następnie dla każdej metody sporządza się wykres błędu względnego w zależności od liczby ewaluacji funkcji (w skali logarytmicznej) oraz analizuje zbieżność i dominację błędu numerycznego.

1.2 Argumentacja rozwiązania

Kwadratury prostokątów, trapezów i Simpsona to podstawowe techniki numerycznego całkowania. Metoda prostokątów wykorzystuje środek każdego podprzedziału, trapezy bazują na polu trapezu pod funkcją, natomiast Simpson przybliża fragmenty funkcji parabolami.

W środowisku SciPy i NumPy zaimplementowano je następująco:

- `scipy.integrate.trapezoid` oraz `scipy.integrate.simpson` dla trapezów i Simpsona.
- Metodę prostokątów zaprogramowano ręcznie, sumując wartości funkcji w punktach środkowych podprzedziałów i mnożąc przez ich szerokość h .

Oto fragment kodu realizujący obliczenia dla podanej całki:

```
1 def f(x):  
2     x = np.float64(x)  
3     return np.float64(4.0) / (np.float64(1.0) + x**2)  
4
```

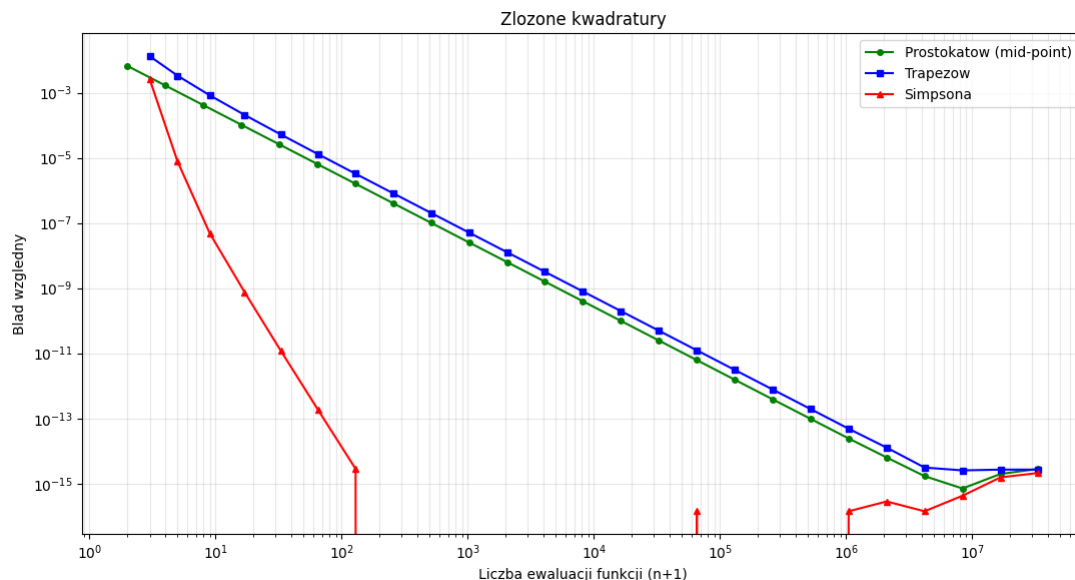
```

5 n = 2**m
6 x = np.linspace(0, 1, n + 1, dtype=np.float64)
7 y = f(x)
8 h = np.float64(x[1] - x[0])
9
10 # Trapezy i Simpson
11 I_trap = np.float64(integrate.trapezoid(y, x))
12 I_simp = np.float64(integrate.simpson(y, x=x))
13
14 # Prostokaty (mid-point)
15 x_mid = (x[:-1] + x[1:]) / np.float64(2.0)
16 I_mid = h * np.sum(f(x_mid))

```

Listing 1: Wyznaczanie całki za pomocą kwadratur złożonych

1.3 Wykresy i wyniki



Wykres 1: Wartość bezwzględna błędu względnego dla metod złożonych w zależności od liczby ewaluacji funkcji ($n+1$).

Dla metody trapezów minimalna liczba ewaluacji wynosi około $8,3 \times 10^6$ (h około $1,19 \times 10^{-7}$), przy której błąd względny to zaledwie $2,54 \times 10^{-15}$.

Rzędy zbieżności w obszarze, w którym błąd metody dominuje nad numerycznym:

- Prostokątów: 1,92 (wartość teoretyczna 2)
- Trapezów: 2,00 (wartość teoretyczna 2)
- Simpsona: 6,50 (wartość teoretyczna 4, natomiast przy specyficznej funkcji i siatce błąd spada bardzo gwałtownie przed wejściem w szum zmiennoprzecinkowy).

1.4 Wnioski

1. Istnieje wartość $h_{\min}$, poniżej której zmniejszanie kroku nie tylko nie poprawia dokładności, ale wręcz ją pogarsza. Wynika to z ograniczonej precyzji obliczeń zmiennoprzecinkowych (`float64`), która daje około 15–16 cyfr znaczących.
2. Przy bardzo dużej liczbie węzłów sumowane jest mnóstwo ułamków, w związku z czym narasta błąd zaokrągleń (błąd numeryczny), przykrywając błąd samej metody (błąd obcięcia).
3. Z wykresu widać, że metoda Simpsona zbiega najszybciej. Jej wykres błędu gwałtownie opada i jako pierwszy uderza w podłogę ustaloną przez podwójną precyzję maszyny, oscylując tam dla większej liczby węzłów.

2 Zadanie 2 – Kwadratura Gaussa-Legendre’a

2.1 Opis zadania

Zadanie polega na obliczeniu tej samej całki przy wykorzystaniu kwadratury Gaussa-Legendre’a. Sprawdzane jest zachowanie błędu względnego dla rosnącej liczby ewaluacji funkcji od 1 do około 30, w celu ukazania momentu napotkania szumu numerycznego.

2.2 Argumentacja rozwiązania

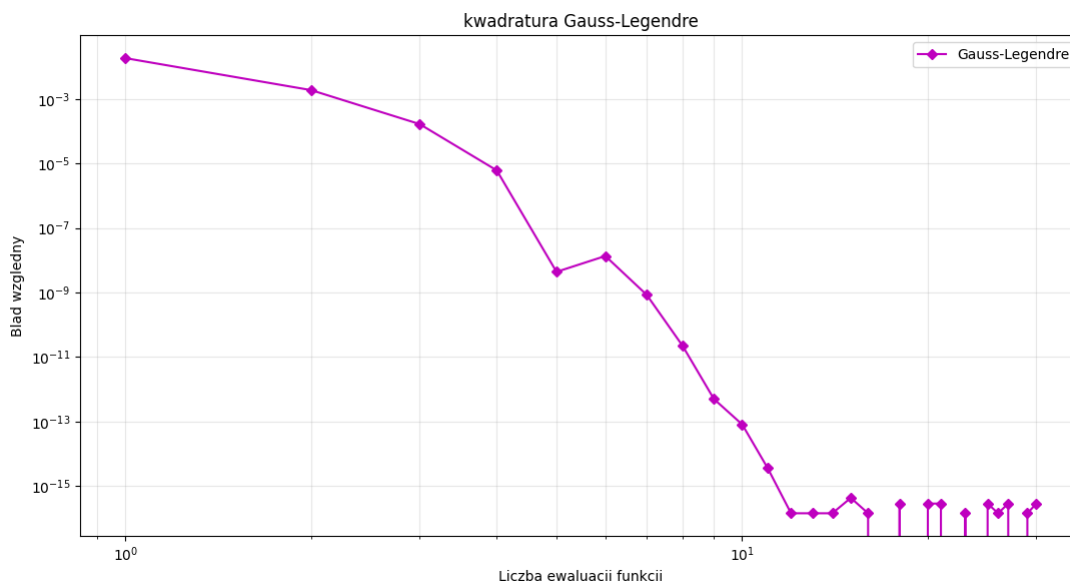
Kwadratura Gaussa-Legendre’a różni się od kwadratur Newtona-Cotesa punktami całkowania. Węzły nie są rozmieszczone równomiernie, lecz odpowiadają pierwiastkom wielomianów Legendre’a, a każdemu węzłowi przypisana jest określona waga. Pozwala to na dokładne całkowanie wielomianów aż do stopnia $2n - 1$ przy zaledwie n węzłach.

Współczynniki pobierano funkcją `leggauss`, a przedział standardowy od -1 do 1 został przekształcony afinicznie na docelowy od 0 do 1 .

```
1 xi, wi = leggauss(n)
2 # Przekształcenie do [0, 1]
3 x_mapped = np.float64(0.5) * (xi + np.float64(1.0))
4 w_mapped = np.float64(0.5) * wi
5
6 I_gl = np.dot(w_mapped, f(x_mapped))
```

Listing 2: Implementacja całki kwadraturą Gaussa

2.3 Wyniki



Wykres 2: Błąd względny dla kwadratury Gaussa-Legendre'a w zależności od liczby punktów obliczeń.

Metoda Gaussa-Legendre'a osiąga doskonałe rezultaty i zbliża się do zera w granicach dokładności maszynowej (`float64`) już przy $n = 17$ punktach.

2.4 Wnioski

1. Kwadratury Gaussa wykazują zbieżność wykładniczą (spektralną) dla funkcji gładkich. W związku z tym już po kilkunastu punktach wykres zanurkował do wartości mniejszych niż 10^{-15} .
2. Ta różnica dobrze pokazuje przewagę kwadratur optymalnych w stosunku do siatek równomiernych. Aby osiągnąć ten sam wynik, metoda trapezów potrzebowała milionów ewaluacji, a Gauss-Legendre zaledwie siedemnastu.

3 Zadanie 3 – Kwadratury adaptacyjne

3.1 Opis zadania

Polecenie wymaga ponownego obliczenia całki za pomocą algorytmów adaptacyjnych: adaptacyjnych trapezów oraz funkcji bibliotecznej `scipy.integrate.quad`, bazującej na adaptacyjnej metodzie Gaussa-Kronroda (QUADPACK). Celem jest zbadanie skuteczności tych metod zadanym tolerancjom błędów i naniesienie wyników na wspólny wykres.

3.2 Argumentacja rozwiązania

Kwadratury adaptacyjne mają za zadanie zagęszczać siatkę ewaluacji tylko w obszarach, gdzie funkcja charakteryzuje się dużą zmiennością (wysoka druga lub wyższa pochodna).

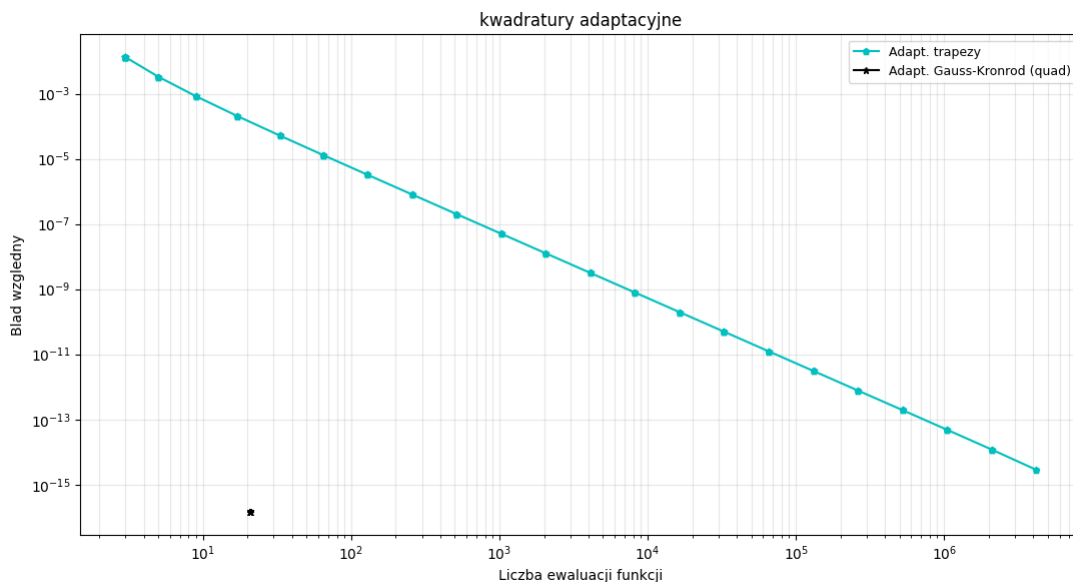
Pozwala to na zoptymalizowanie czasu procesora i zminimalizowanie potrzebnych obliczeń w miejscach płaskich.

Adaptacyjna kwadratura trapezów jest implementowana ręcznie poprzez stopniowe podwajanie gęstości, do momentu kiedy szacowany błąd pomiędzy starym a nowym krokiem będzie akceptowalny względem narzuconej tolerancji (od 1 do 10^{-14}). Do metody Gaussa-Kronroda wykorzystano dedykowaną funkcję ze zmienną `epsabs` równej ustalonej tolerancji.

```
1 quad_out = integrate.quad(f, 0, 1, epsabs=tol, epsrel=tol, full_output=True)
2 res_gk = np.float64(quad_out[0])
3 n_evals_gk.append(quad_out[2]['neval'])
```

Listing 3: Wywołanie funkcji `quad` do Gaussa-Kronroda

3.3 Wyniki



Wykres 3: Osiągany błąd kwadratur adaptacyjnych względem liczby ewaluacji dla różnych tolerancji.

Dla algorytmu `quad` (Gauss-Kronrod) do osiągnięcia dokładności rzędu błędu maszynowego wykorzystuje się tylko kilkadziesiąt ewaluacji funkcji. Wariant adaptacyjnych trapezów naturalnie staje się precyzyjniejszy dużo wolniej ze względu na niski rząd funkcji bazowej (trapezu).

3.4 Wnioski

1. Biblioteka `scipy.integrate.quad` doskonale optymalizuje obliczenia, ponieważ jest bardzo efektywna dla całek oznaczonych. Jest to hybryda zaawansowanych estymacji błędu Kronroda ze znakomitą zbieżnością węzłów Gaussa.
2. Algorytmy adaptacyjne są nieocenione w inżynierii, jeśli funkcja cechuje się punktowymi gwałtownymi wzrostami i zależy nam na oszczędzeniu mocy obliczeniowej.

4 Zadanie 4 – Metoda Monte-Carlo

4.1 Opis zadania

Zadanie zakłada odnalezienie wartości całki numerycznej za pomocą losowych metod probabilistycznych Monte-Carlo. Rysowany jest wykres błędu względem bardzo dużej liczby generowanych próbek losowych n .

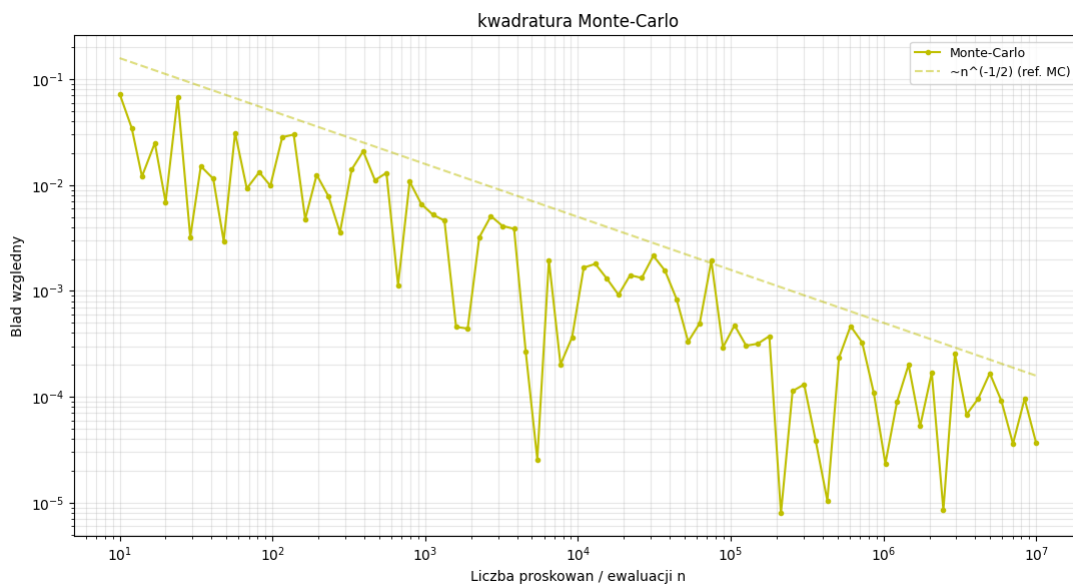
4.2 Argumentacja rozwiązania

Zgodnie z metodą wartości średniej Monte-Carlo, przybliżenie całki można wyrazić jako wartość oczekiwaną funkcji w podanym przedziale. Dla przedziału $[0, 1]$ i dużej próby N , wartość ta zbiega wprost proporcjonalnie do średniej arytmetycznej.

$$I \approx \frac{b-a}{N} \sum_{i=1}^N f(X_i)$$

W bibliotece NumPy tablicę N liczb losowych w przedziale $(0, 1)$ wygenerowano, wykorzystując `np.random.uniform`.

4.3 Wyniki



Wykres 4: Zbieżność błędu całkowania metodą Monte-Carlo od liczby próbek N .

Metoda działa poprawnie i po pewnym czasie zbliża się do prawdziwej wartości π , czyli na przestrzeni kilkunastu milionów ewaluacji, błąd zmniejsza się w sposób liniowy w skali logarytmicznej.

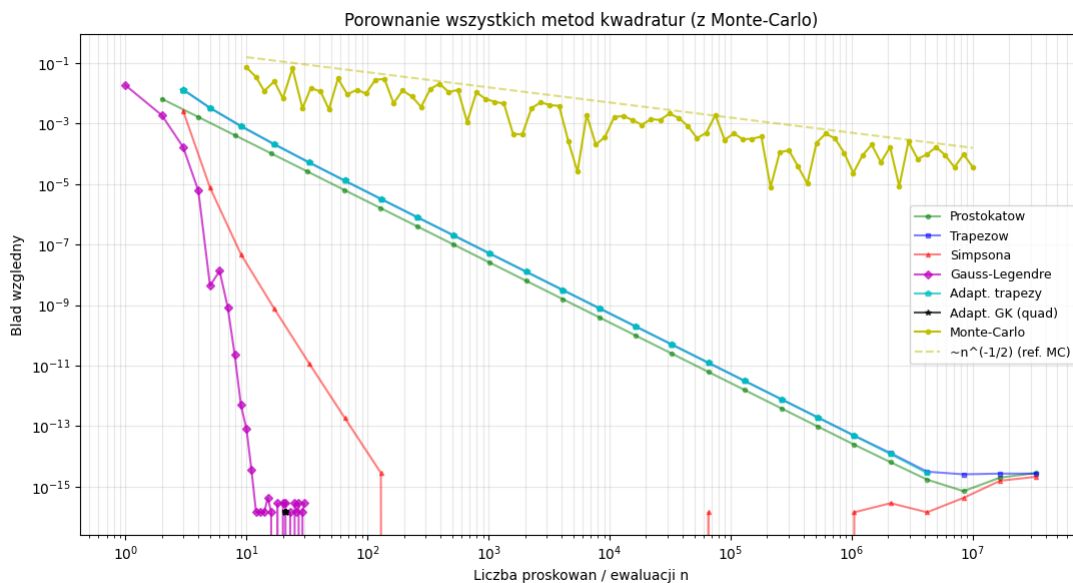
4.4 Wnioski

1. Zbieżność metody Monte-Carlo jest bardzo niska i uzależniona jest ściśle od wartości pierwiastka kwadratowego liczby prób $\mathcal{O}(\frac{1}{\sqrt{N}})$.

2. Na wykresie błędu doskonale widać lokalne duże losowe fluktuacje wywołane stochastycznym charakterem pobierania punktów. Gwarancja bliskości do celu wywodzi się jedynie z Centralnego Twierdzenia Granicznego.
3. Choć jest mało wydajna do prostych całek jednowymiarowych, staje się świetnym wyborem podczas rozwiązywania problemów całkowania po przestrzeniach wielowymiarowych o dużym skomplikowaniu (tzw. kłątwa wymiarowości), w których standardowe siatki kwadratur musiałyby uwzględnić wręcz ogromną liczbę operacji.

5 Podsumowanie całościowe

5.1 Porównanie wszystkich badanych metod



Wykres 5: Zestawienie błędu względnego wszystkich analizowanych metod całkowania w zależności od liczby ewaluacji.

Powyższy wykres zbiorczy doskonale ukazuje różnice w szybkości zbieżności poszczególnych algorytmów numerycznych w jednym ujęciu. Zestawienie to dobrze obrazuje różnicę pomiędzy podstawowymi metodami opartymi na równomiernej siatce a zaawansowanymi technikami optymalnymi.

Można zauważyć, że kwadratura Gaussa-Legendre'a oraz adaptacyjna metoda Gaussa-Kronroda (**quad**) niemal natychmiastowo zbiegają do poziomu szumu maszynowego. Złożone metody Newtona-Cotesa również osiągają dobre rezultaty i ostatecznie zderzają się z błędem zmiennoprzecinkowym, jednakże potrzebują do tego znacznie więcej ewaluacji badanej funkcji. Z kolei krzywa błędów metody Monte-Carlo, ze względu na bardzo powolny proces zbierania probabilistycznego, znajduje się znacznie wyżej na wykresie i maleje z ledwie zauważalnym nachyleniem, co pokazuje, że jest ona mało wydajna w przypadku łatwych całek jednowymiarowych.

5.2 Główne wnioski

1. **Cena metody Newtona-Cotesa.** Podstawowe metody jak prostokąty czy trapezy opierają się na gęstej, stałej siatce. Dla wymagających problemów natrafimy w nich na ścianę w postaci błędu numerycznego obliczeń float, która zatrzyma zbieżność przy pewnej wartości h_{min} .
2. **Przewaga Gaussa.** Kwadratura Gaussa-Legendre'a daje znacznie lepsze wyniki niż klasyczne rozwiązania w całkach jednowymiarowych z funkcji gładkich pod kątem rzędu zbieżności i czasu obliczeń. Pozbywa się problemu narzucanych węzłów wybierając optymalne dla przybliżeń wielomianowych.
3. **Zastosowanie.** Dobór odpowiedniego narzędzia zależy od dziedziny całkowania i specyfiki funkcji. `SciPy` oferuje rozwiązanie idealne dla większości uniwersalnych gładkich całek za pomocą adaptacyjnego QUADPACK (`quad`), optymalizując lokalnie rozmieszczenie kroków.